

Question 7 Forecasting Pipeline: Flow and Components

How data moves from raw food-shelf records to the August 2026 county submission

MinneMUDAC 2026 Undergraduate Team

Information cutoff: July 31, 2026

This document describes the *structure* of the forecasting pipeline in `forecast_august_2026/`: its stages, the scripts that implement them, what each one reads and writes, where the information cutoff is enforced, and how the final model turns county history into a forecast. The companion report (`forecasting_report.pdf`) contains the results.

1 End-to-end flow

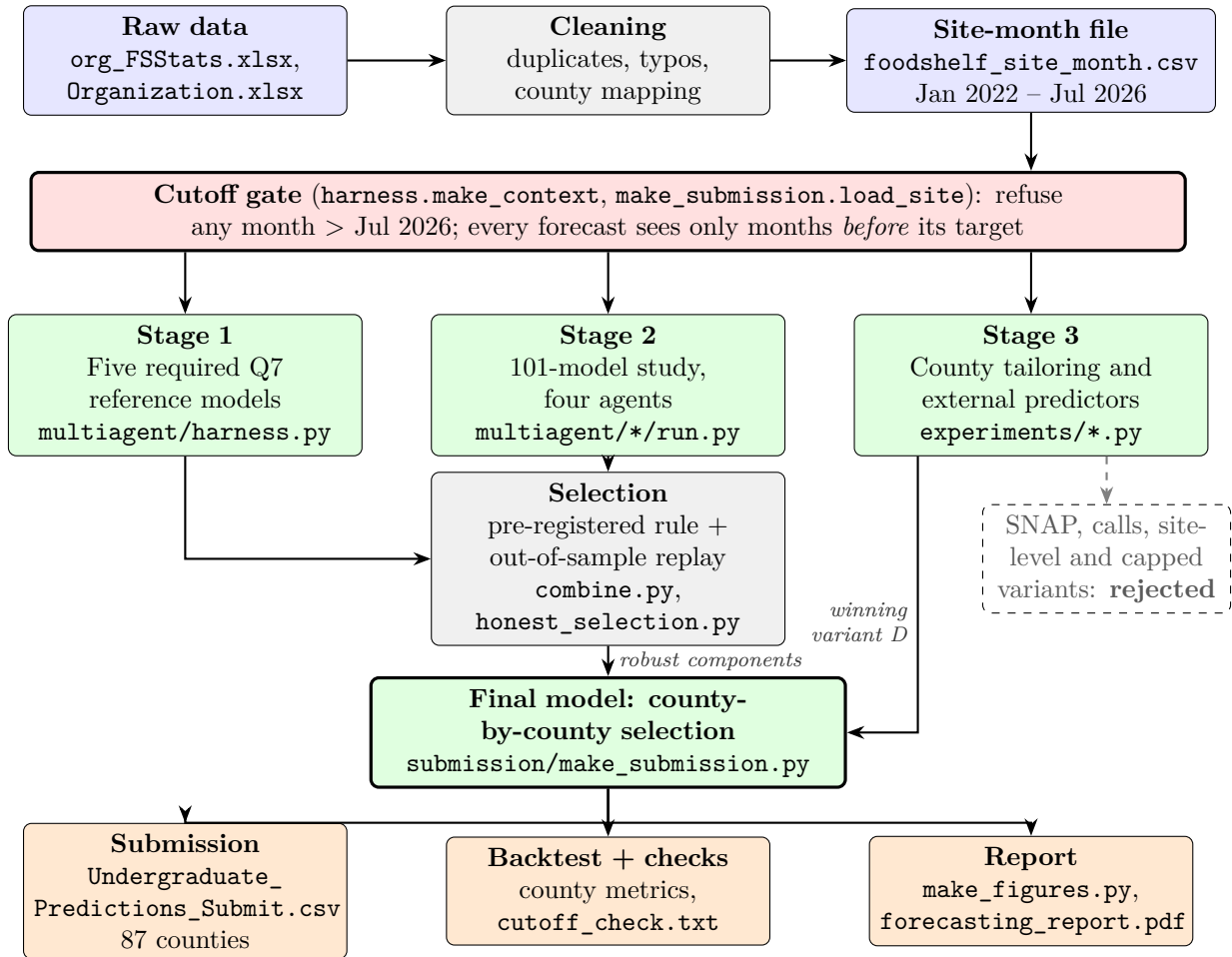


Figure 1: End-to-end pipeline. Blue: data; red: cutoff enforcement; green: modelling stages; grey: selection; orange: outputs; dashed: tested and rejected.

The pipeline has one data source for the submitted model (the prepared site-month file), one place where the cutoff is enforced for all modelling (the cutoff gate), three modelling stages that progressively narrow the choice of method, and one final script that produces every submission artifact.

2 Stage by stage

Table 1: Pipeline stages, scripts, inputs, and outputs (paths relative to `forecast_august_2026/`).

Stage	Script(s)	Reads	Writes / purpose
0. Data	project cleaning notebooks	raw Excel exports	Data/processed/foodshelf_site_month.csv : one row per site and month, duplicates and typos resolved, outlier flags kept
Gate	<code>multiagent/harness.py</code> <code>submission/make_submission.py</code>	site-month file	Slices data to months before each target; asserts the slice ends exactly one month earlier; refuses any month after July 2026
1. Reference models	<code>multiagent/harness.py</code>	gate output	Seasonal naive, YTD growth, July $\times$ Aug/Jul ratio, trend + month regression, robust ensemble, damped YoY $\rightarrow$ baselines/results/
2. Model study	<code>multiagent/stats/run.py</code> <code>multiagent/panel_ml/run.py</code> <code>multiagent/deep_foundation/run.py</code> <code>multiagent/structural/run.py</code>	gate output	101 variants (ETS, ARIMA, Theta, LightGBM, NHITS, Chronos, ...) $\rightarrow$ each agent's results/predictions.csv and REPORT.md
Selection	<code>multiagent/combine.py</code> <code>multiagent/honest_selection.py</code>	all agents' predictions	Leaderboards, pre-registered rule, out-of-sample replay of model picking. Outcome: keep simple robust components
3. Experiments	<code>experiments/county_tailoring.py</code> <code>experiments/external_predictors.py</code>	gate output, SNAP files, calls	Compares county-tailoring variants and external predictors; chooses county-by-county selection, rejects SNAP and calls
Final model	<code>submission/make_submission.py</code>	site-month file, submission template	Submission CSV, per-county components and choices, county backtest, statewide totals with 80% ranges, cutoff_check.txt
Report	<code>report/make_figures.py</code> <code>report/forecasting_report.tex</code>	all results above	Figures and the written report

3 How every forecast is evaluated: the backtest loop

All stages share the same evaluation loop. The key property is the order of operations: the model produces its forecast *before* the actual value is looked up.

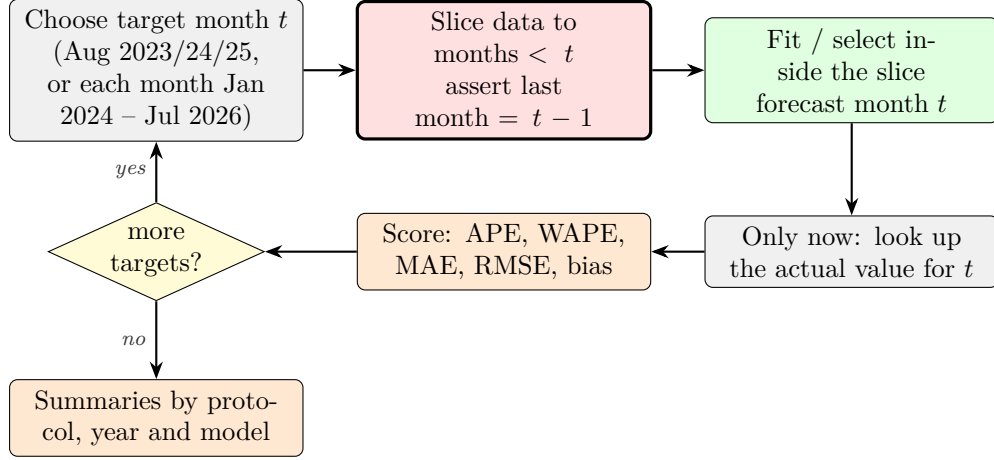


Figure 2: The backtest loop used in every stage. Protocol A: the three August folds. Protocol B: 31 one-month-ahead origins.

Any tuning a model needs (damping factor, window length, local-model choice, network weights) happens inside the “fit / select” box, using only the slice. No parameter is tuned against the backtest scores themselves.

4 Inside the final model

For August 2026 the final script repeats the same steps for each of the three outcomes (visits, pounds, individuals) and each county.

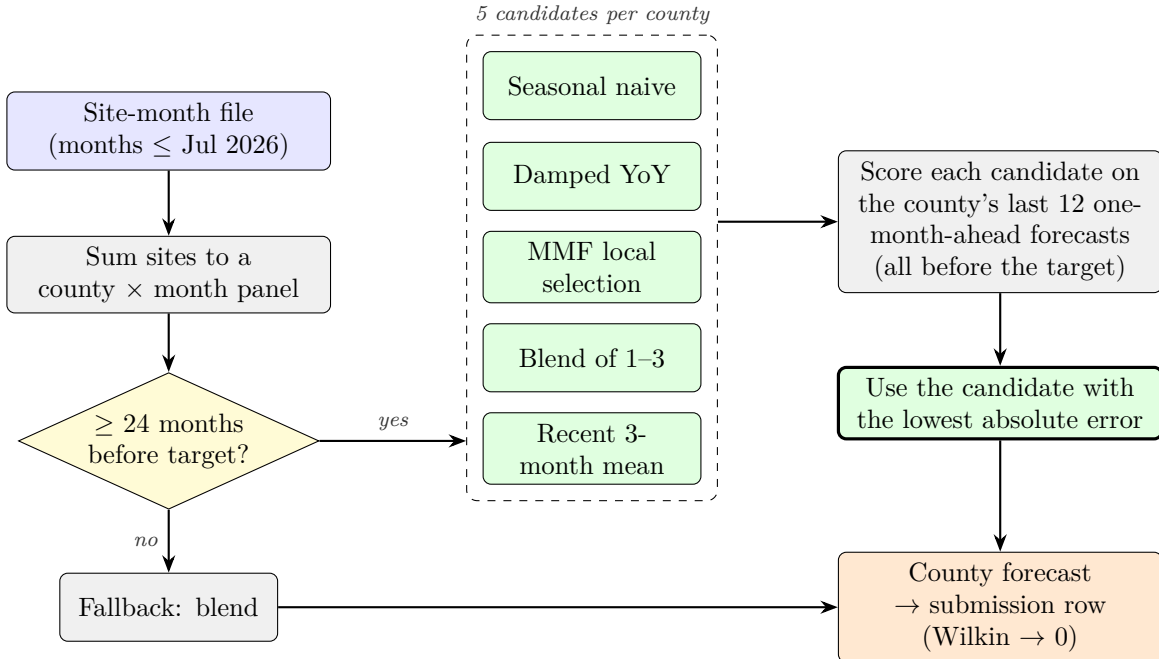


Figure 3: How the final model forecasts one county. Counties with a recent level shift (for example a new site) tend to choose the recent mean; stable counties keep the blend or seasonal naive.

Table 2: The five candidates the final model chooses from, for a target month t .

Candidate	Definition
Seasonal naive	$\hat{y}_t = y_{t-12}$
Damped 3-month YoY	$\hat{y}_t = y_{t-12} (1 + 0.5 (g - 1))$, where $g = \sum_{k=1}^3 y_{t-k} / \sum_{k=1}^3 y_{t-12-k}$
MMF local selection	whichever of {seasonal naive, damped YoY, full YoY, recent mean, recent level $\times$ last year’s seasonal shape} had the lowest error over the last 6 months
Blend	mean of the three candidates above
Recent 3-month mean	$\hat{y}_t = \frac{1}{3} \sum_{k=1}^3 y_{t-k}$

For August 2026 the visits forecasts used the recent mean in 33 counties, damped YoY in 21, the blend in 14, seasonal naive in 10, and MMF selection in 8.

5 Where the cutoff is enforced

Table 3: Cutoff checks and where they live.

Check	Implementation
No source month after July 2026	<code>harness.load_site</code> , <code>make_submission.load_site</code> : assertion on the maximum month
Training slice ends at $t - 1$	<code>harness.make_context</code> , <code>make_submission.county_panel</code> : assertion on every call ($\sim 10,000$ calls across the study)
Actuals looked up after forecasting	<code>harness.evaluate</code> , <code>make_submission.backtest</code> : actual retrieved only after the forecast is stored
Selection uses only earlier origins	<code>make_submission.county_select</code> : the 12 scoring origins all lie before the target
External data lagged	<code>experiments/external_predictors.py</code> : SNAP and calls only from at least 2–5 months before the target; the SNAP dashboard is truncated at July 2026
Audit trail	every run writes <code>cutoff_check.txt</code>

6 Folder layout and run order

Folder layout

```
forecast_august_2026/
  README.md
  submission/          <- final model
    make_submission.py
    submission_template.csv
    Undergraduate_Predictions_Submit.csv
    county_backtest_*.csv
    statewide_forecast.csv
    cutoff_check.txt
  multiagent/          <- stages 1-2
    harness.py
    SELECTION_RULE.md
    combine.py
    honest_selection.py
    baselines/ stats/ panel_ml/
    deep_foundation/ structural/
  experiments/          <- stage 3
    county_tailoring.py
    external_predictors.py
    results/
  report/
    make_figures.py
    forecasting_report.tex/.pdf
    pipeline_flow.tex/.pdf
```

Run order (from forecast_august_2026/)

1. `python multiagent/harness.py`
reference models (≈ 2 s)
2. `python multiagent/<agent>/run.py`
optional; 2–20 min each, results already saved
3. `python multiagent/combine.py`
leaderboards and selection rule
4. `python experiments/county_tailoring.py`
chooses the final model
5. `python experiments/external_predictors.py`
SNAP and calls tests
6. `python submission/make_submission.py`
`--team-id ID`
final submission (≈ 1 min)
7. `python report/make_figures.py`
then compile the two `.tex` files

Only step 6 is needed to regenerate the submission; steps 1–5 reproduce the evidence behind it.

7 Design decisions in one place

- **One data source, one gate.** The submitted model reads only the prepared site-month file, and every model goes through the same cutoff-checked slicing.
- **Same horizon everywhere.** All tests are one month ahead, matching the real task (July data $\rightarrow$ August forecast).
- **Two evaluation protocols.** Three August folds (the official question) plus 31 monthly origins (enough points to separate real gains from noise).
- **Simple components, adaptive selection.** Advanced models and external predictors were tested and rejected on out-of-sample evidence; the gain comes from letting each county choose among robust methods based on its own recent track record.
- **Everything reproducible.** Each table and figure has a script, and each run leaves a cutoff audit.